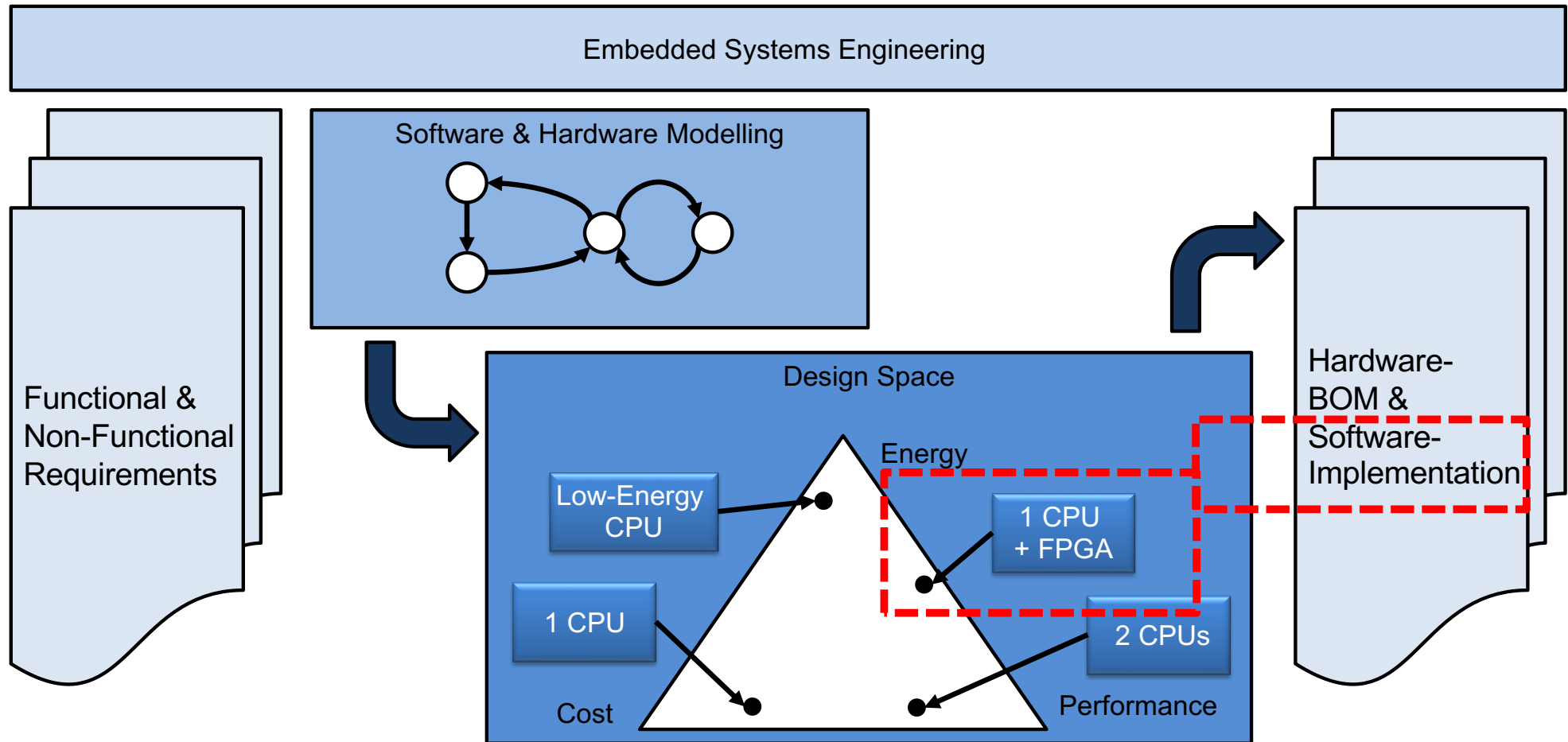


Embedded Software Engineering

Introduction to FPGA

Simon Künzli, Tobias Welti
HS 2025

Advance Organizer ESE



Goals

After today's lesson, you should be able to ...

- ... explain the basic principle of an FPGA.
- ... explain how logic functions are implemented in an FPGA.
- ... identify functions in your embedded software that could be accelerated in an FPGA
- ... judge whether an algorithm can be put in an FPGA efficiently.
- ... understand how software in the CPU interacts with an FPGA accelerator
- ... participate in a discussion on whether an FPGA should be considered for your project

What's an FPGA to you?

■ [www.menti.com > 5750 0715](https://www.menti.com/join/57500715)



Applications for FPGA



MMC Studios

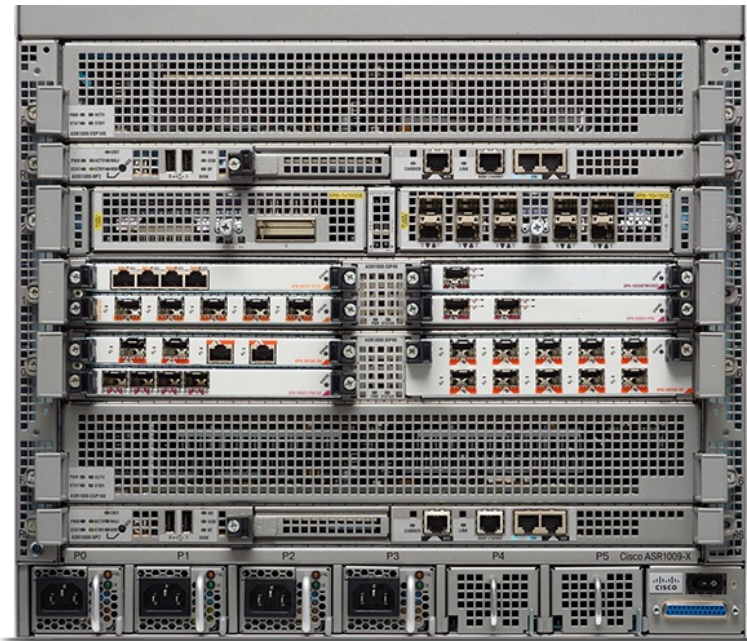
Studio Equipment



Teledyne Lecroy

Oscilloscopes

Applications for FPGA



Cisco

Networking



Philips

Medical Imaging

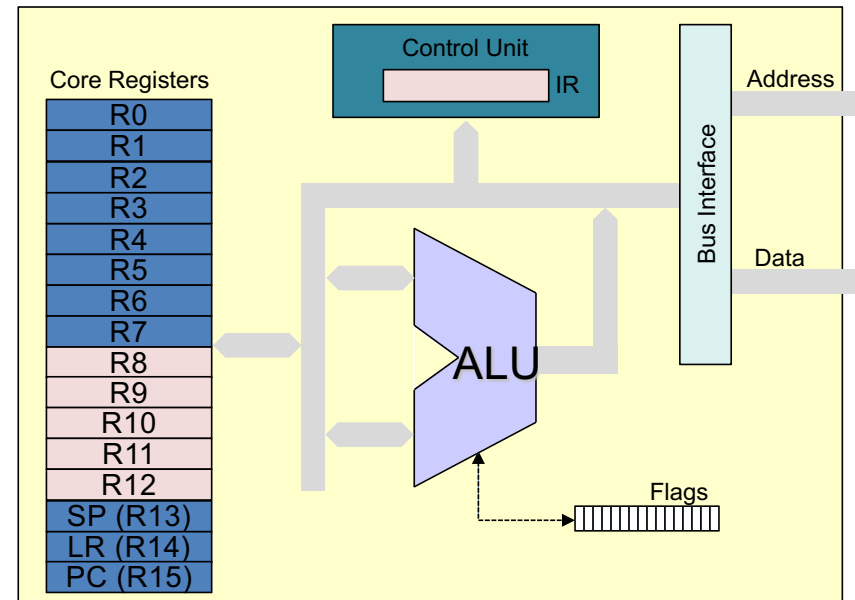
Common requirements of applications?

What requirements do FPGA applications have?

- **high data throughput**
- **highly independent parallelism**
- **deterministic execution time**
- **low latency**
- **many interfaces or data streams**

■ CPU Components

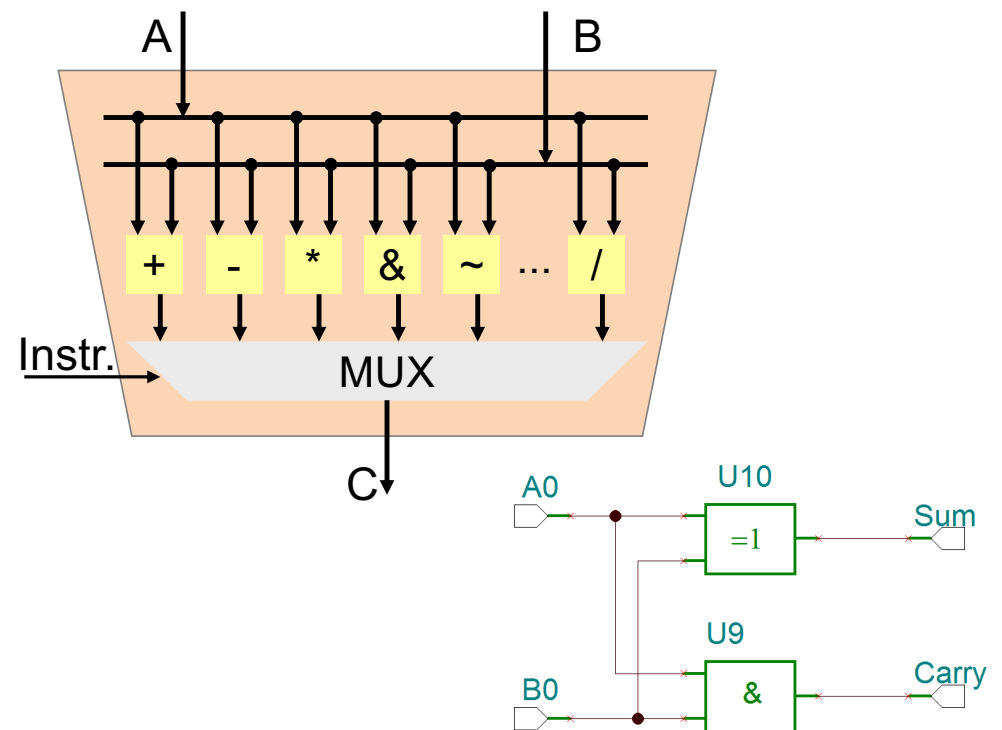
- Core Registers
- 32-bit ALU
- Flags (APSR)
- Control Unit with IR (Instruction Register)
- Bus Interface



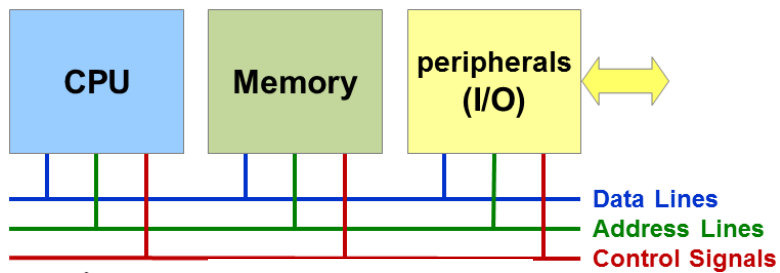
simple CPU model based on the Programmers'
Model of the ARM Cortex-Mx CPUs ¹⁾

■ ALU – Arithmetic Logic Unit

- 32-bit wide data processing unit
 - inputs A and B
 - result C
- integer arithmetic
 - addition / subtraction
 - multiplication / division
 - sign extension
- logic operations
 - AND, NOT, OR, XOR
- shift/rotate
 - left / right



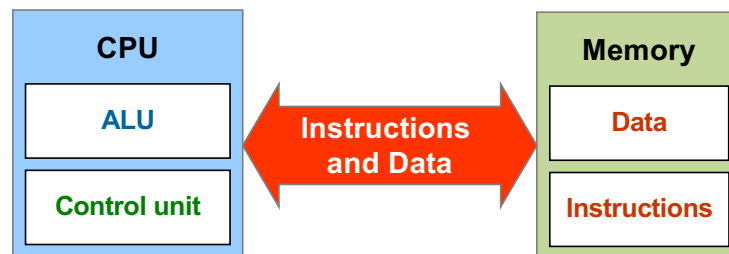
■ More components...



CPU...

- ... fixed in silicon
- ... optimized for general purpose
- ... has a defined Instruction Set

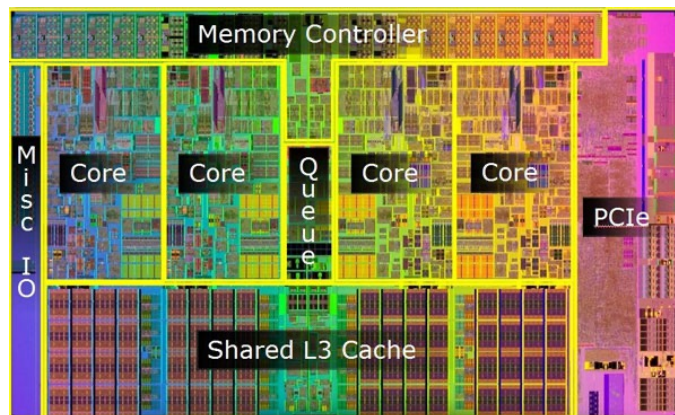
von Neumann Architecture



CPU vs GPU

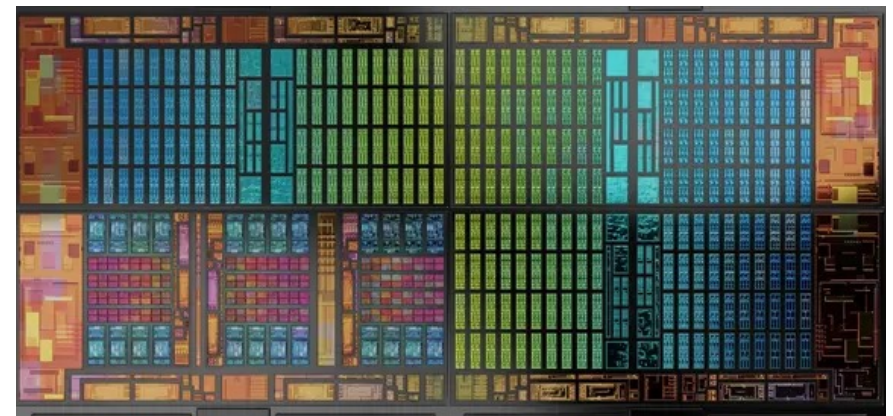
CPU

- Single- or multi-core
- General purpose, many languages
- Software easy to write
- Compilers take care of optimization
- OS handles parallelism and synchronization



GPU

- Highly parallel execution
- Dedicated programming language
- Needs knowledge about hardware to write efficient software
- Synchronization and parallelism handled by driver

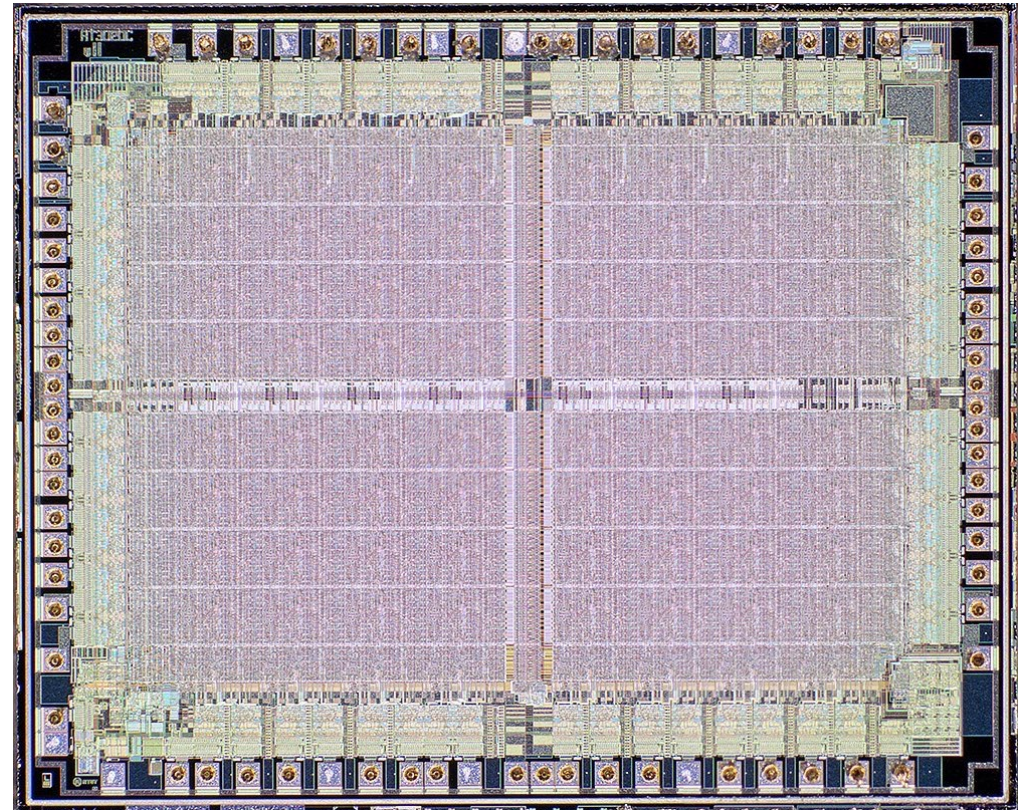


AMD Instinct MI300

FPGA Field Programmable Gate Array

- Programmable logic cells
- Programmable connections between logic cells
- No ALU, System Bus, or CPU
- No Instruction Set

- How do we implement...
 - ... a logic function/arithmetic?
 - ... flow control?
 - ... data transfer?

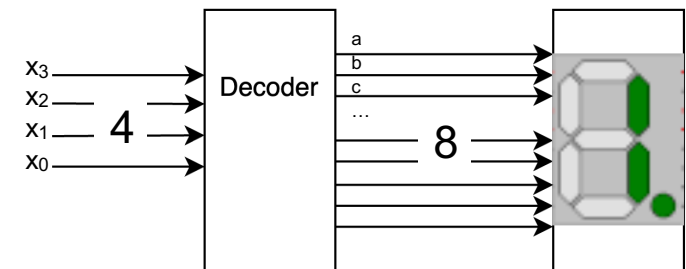


Logical functions: Truth Tables

Seven-segment decoder

- Four inputs, eight outputs
- Input: binary coded decimal
- Output: on/off signal for eight segments
- Behaviour defined by truth table:

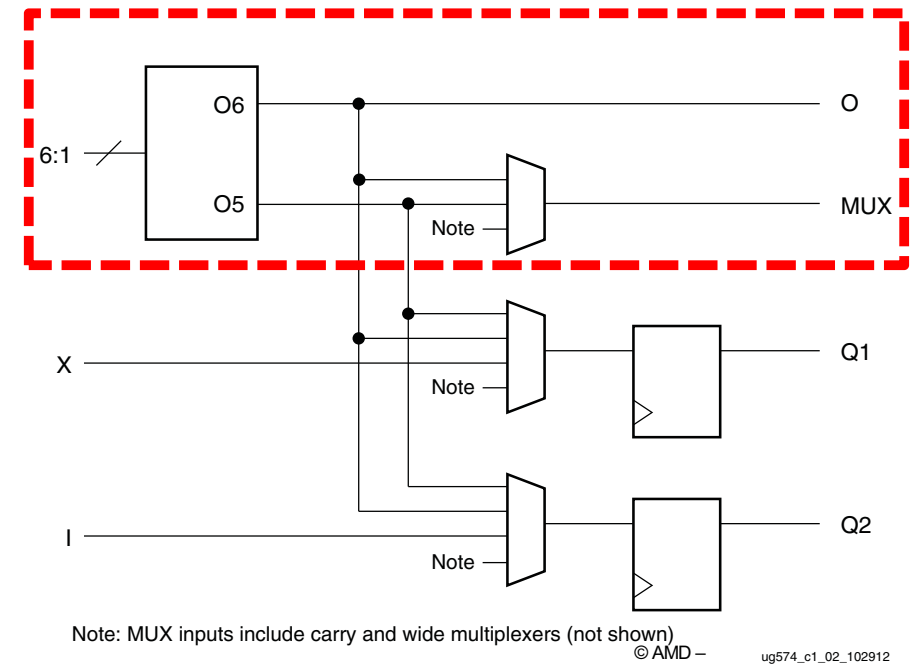
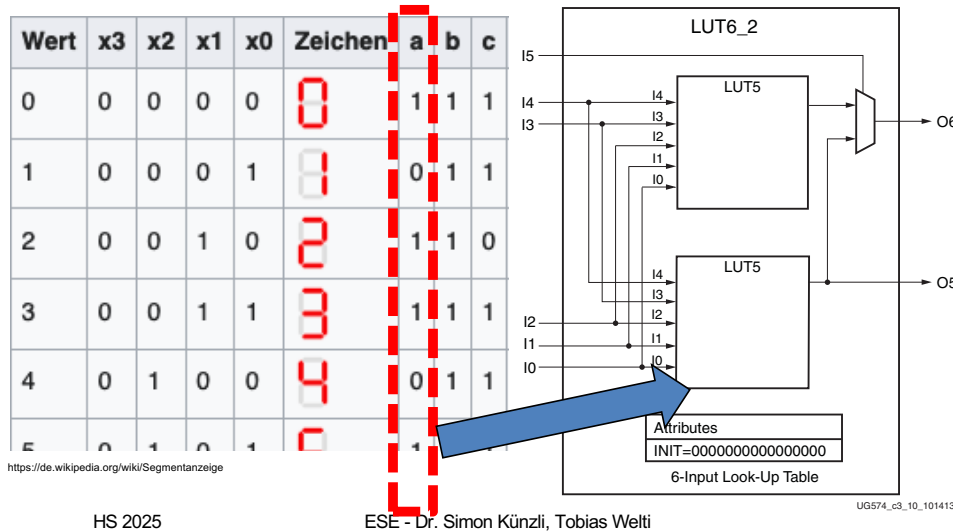
Wert	x3	x2	x1	x0	Zeichen	a	b	c	d	e	f	g
0	0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	1	0	1	1	0	0	0	0
2	0	0	1	0	2	1	1	0	1	1	0	1
3	0	0	1	1	3	1	1	1	1	0	0	1
4	0	1	0	0	4	0	1	1	0	0	1	1
5	0	1	0	1	5	1	0	1	1	0	1	1



Configurable Logic Block

Logic Gates

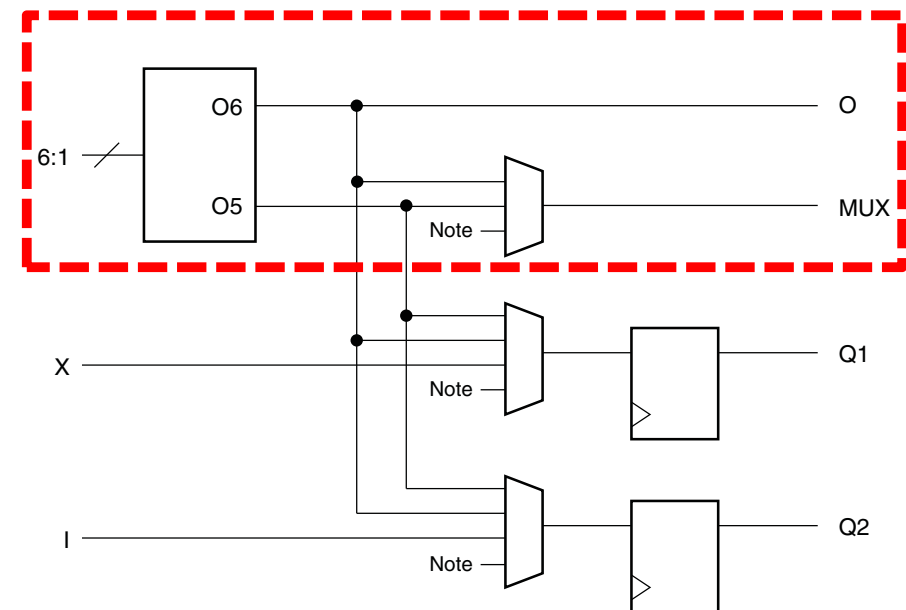
- Implemented as Lookup Table
- Input acts like an address, selects output
- LUT stores output for every input combination



Configurable Logic Block

Arithmetic

- Can be implemented as a lookup table.
- Implements a full adder for one bit (carry and sum).
- Cascade multiple LUTs for larger bit widths.
- The LUT approach is slow.
- "Hard" adders and multipliers are available, fixed in silicon as in a CPU.



Note: MUX inputs include carry and wide multiplexers (not shown)

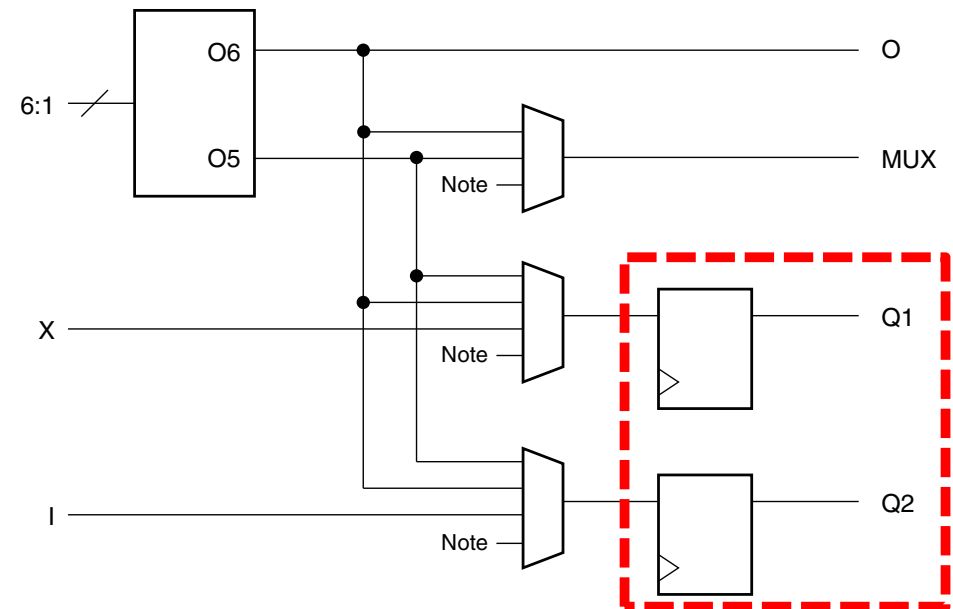
© AMD –

ug574_c1_02_102912

Configurable Logic Block

Memory cells

- Flip Flop stores one bit
- Clock input → sequential logic such as state machines, counters, shift registers



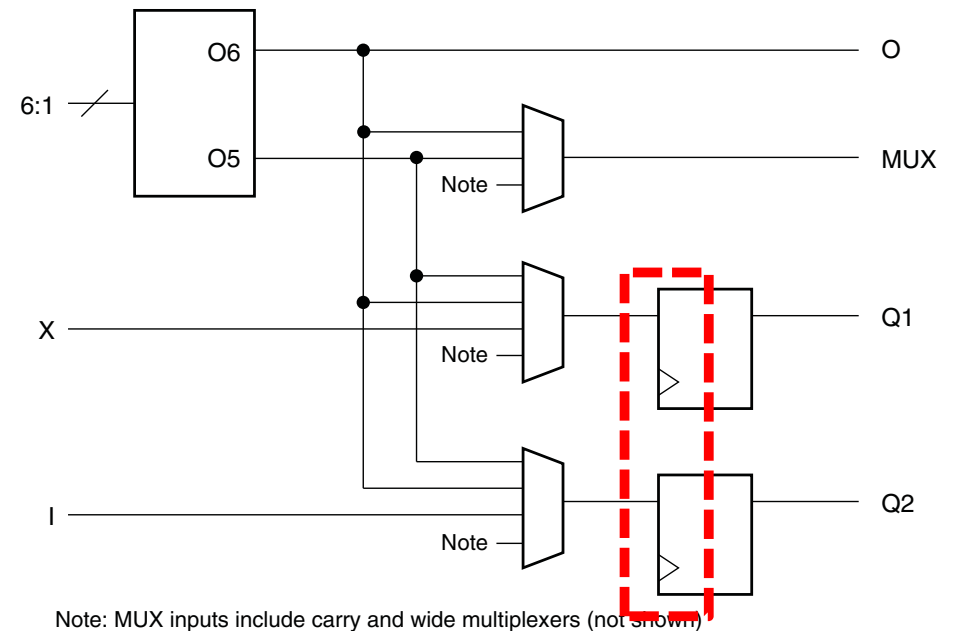
Note: MUX inputs include carry and wide multiplexers (not shown)

© AMD – ug574_c1_02_102912

Clocking

Flip Flops need a clock

- Generated on-chip in a configurable PLL
- Distributed via dedicated routing
- Clock speed limits depend on:
 - device family (50-1000 MHz)
 - complexity of logic functions
 - routing
- Clock speed can be further limited:
 - by other blocks than CLB:
 - on-chip RAM blocks (600 MHz),
 - dedicated Multipliers (400 MHz),
 - ...
 - power limit (higher clock speed uses more power)

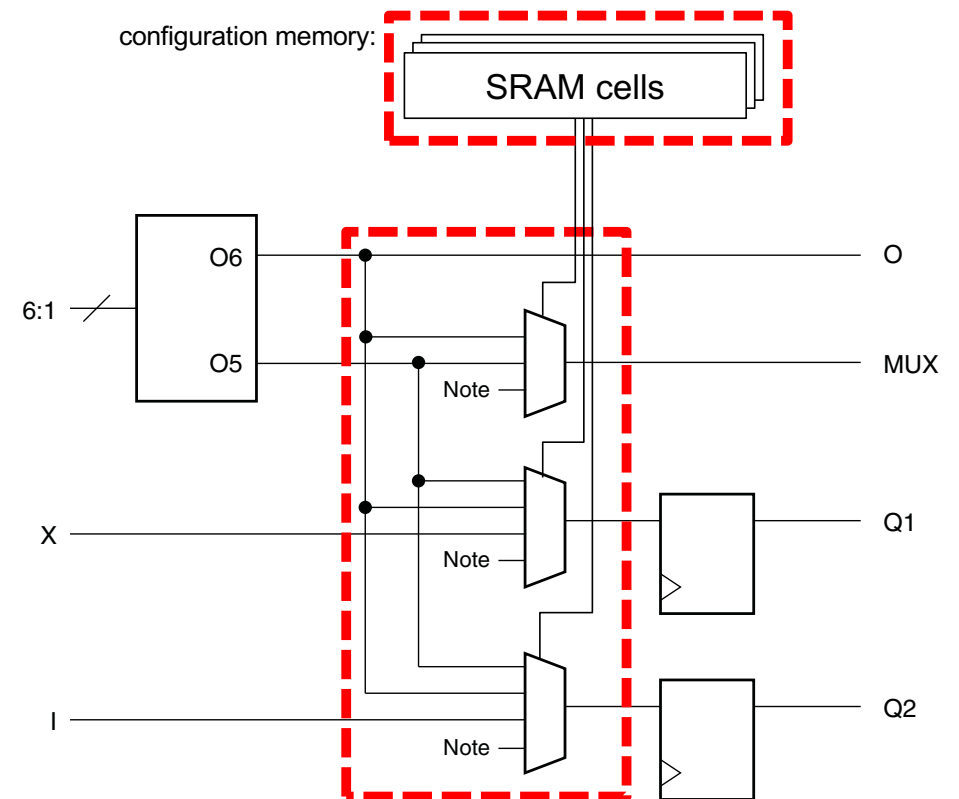


© AMD – ug574_c1_02_102912

Configurable Logic Block

Connections

- Implemented as Multiplexers
- Connections are user-configurable by setting configuration bits
- SRAM- or Flash cells drive 'Select'-Input to Multiplexers to define the connections



Note: MUX inputs include carry and wide multiplexers (not shown)

© AMD –

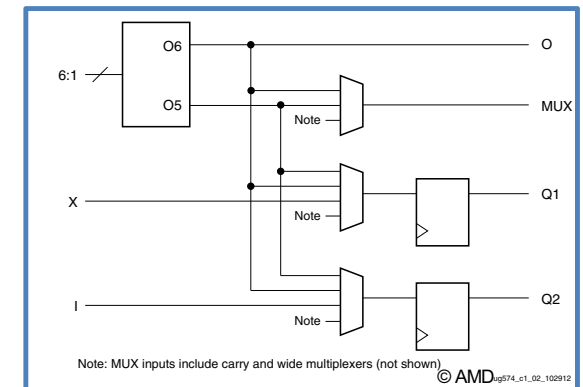
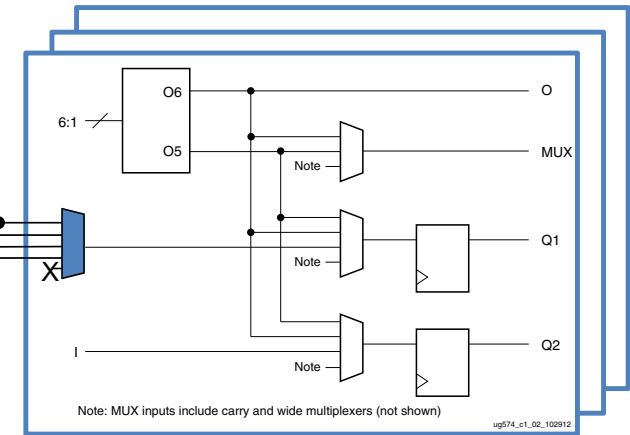
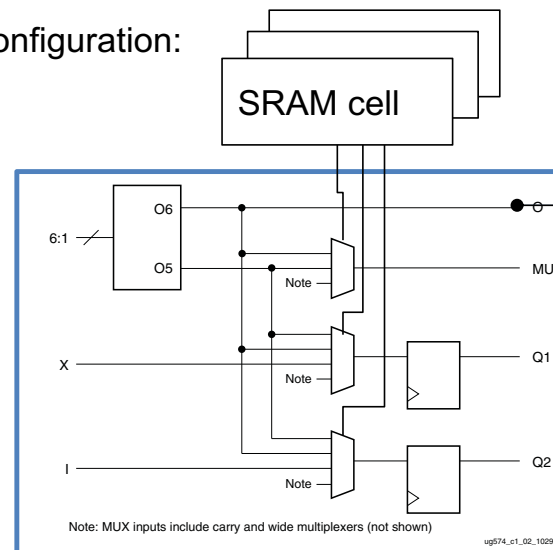
ug574_c1_02_102912

Routing: connect multiple CLB

Combine CLBs to form more complex functions

- configurable connections between CLBs allow flexible routing
- MUX select a routing lane (onramp, offramp)

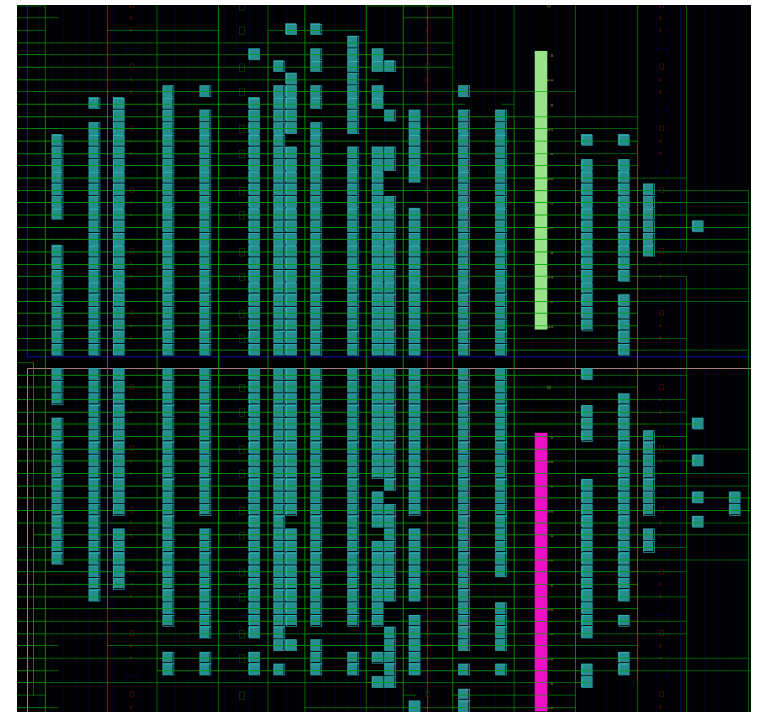
Configuration:



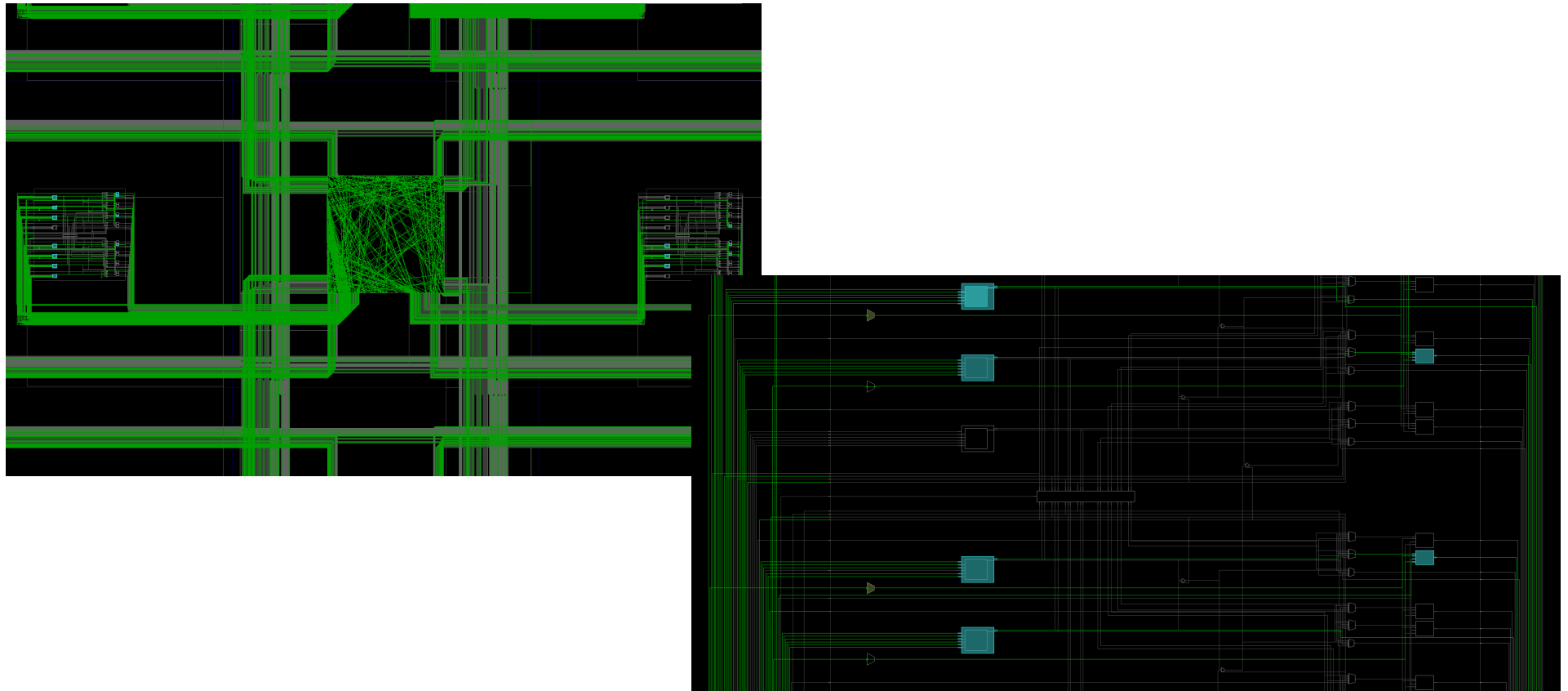
Routing: connect multiple CLB

Building more complex functions

- Cascading/interconnecting CLBs allows larger LUTs for complex logic functions
- Flip Flops can be set up
 - in series to build shift registers
 - in parallel to store larger bit patterns
- combine many CLB via configurable connections:
→ **Routing**
- **Routing** is also configured via multiplexers that choose the connection to the next CLB



Routing



FPGA Pro and Contra

Pro

- Reconfigurability: update your processor/device in the field
- high-speed and parallel processing
- Algorithms can be accelerated, parallelized
- Offloading of functions from CPU
- Very flexible
- Ideal for prototyping of a processor
- Ideal in applications with limited quantities

Contra

- Price tag: more expensive than a microcontroller or an ASIC
- Slower and less efficient than an ASIC
- Often more development time than SW
- Lower clock frequencies than ASIC or CPU
- “Difficult” to work with: You need to know your hardware, and it takes more experience than software development
- Math operations can be cumbersome: **Multiplication, Division, Floating Point** need many logic cells, or specific multipliers in silicon

Applications of FPGA

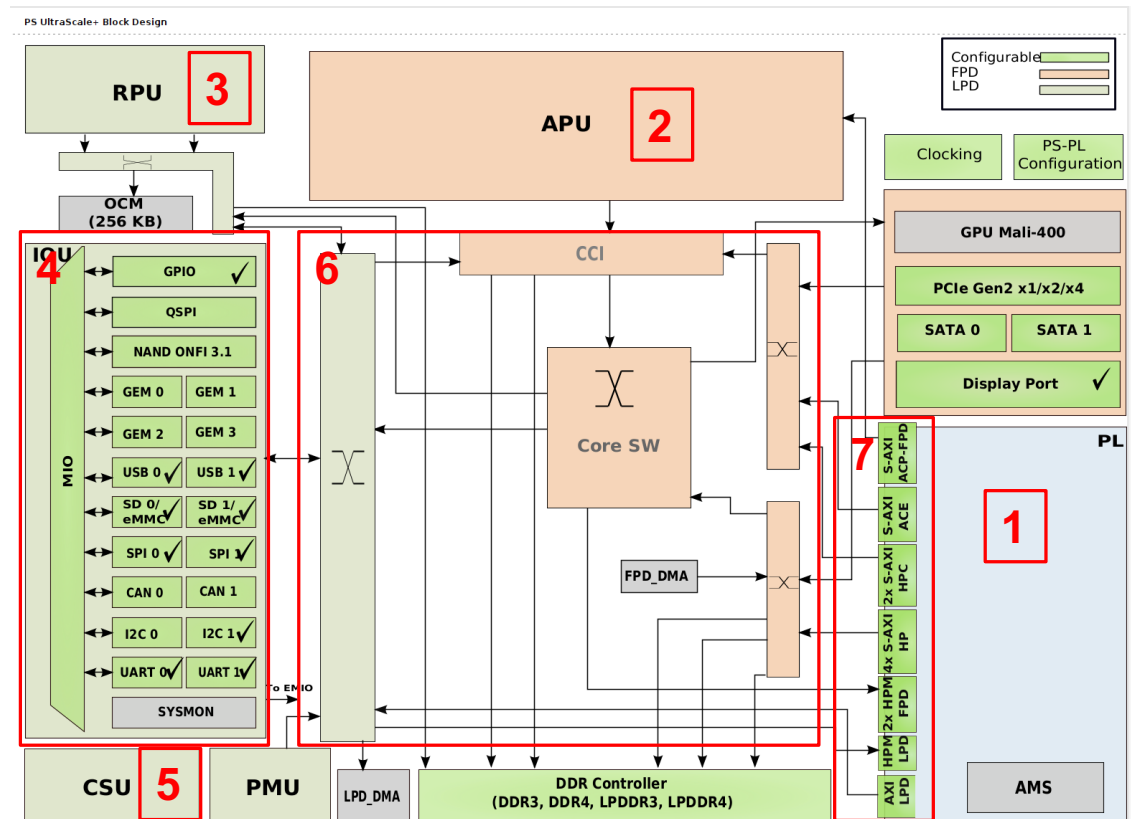
- Search algorithms: Bing.com runs on FPGA accelerators (highly customized algorithm, distributed memory, fast parallel networking)
- General acceleration in data centers: largest customers of FPGA suppliers are data centers
- Real-time image and signal processing:
 - Oscilloscopes
 - Video Testing Devices
 - Software-defined radio
- High-speed interface design
(i.e., 5G-networks, 5G antenna controller for MIMO beamforming, ...)
- Smart cameras: image/video analysis in the camera, close to the sensor
(improved privacy, less network traffic, ...)

System on Chip: FPGA “plus”

- FPGA applications might need software to control accelerated algorithms
 - i.e. video processing configuration parameters better calculated in software
- “Softcore”: configure a small, slow processor in the FPGA fabric
 - microblaze II (AMD)
 - NIOS II (Intel)
 - new option: RISC V → “NIOS V” as softcore
- “Hardcore”: Add processors to the silicon design, as on the ultra96.

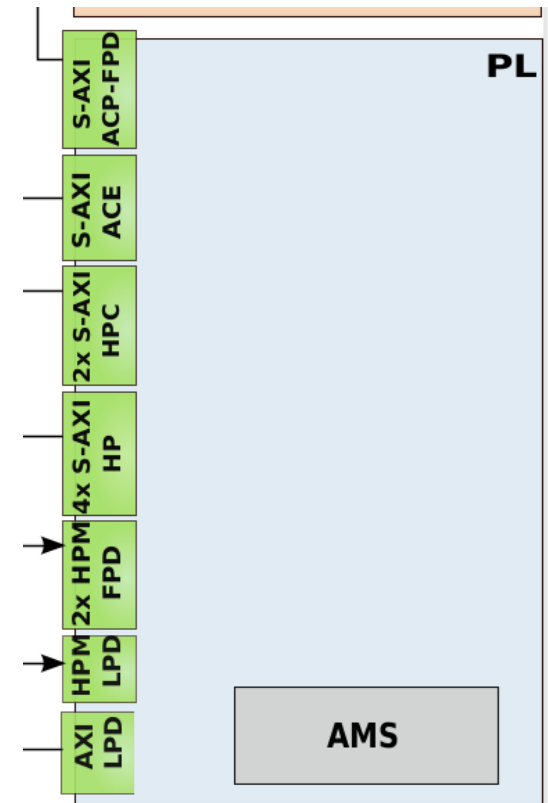
System on Chip Overview

1. **FPGA Fabric**
2. **Application processors**
3. **Real-time processors**
4. **Peripherals**
5. **Configuration/Security Unit**
6. **System Bus**
7. **AXI Interfaces**



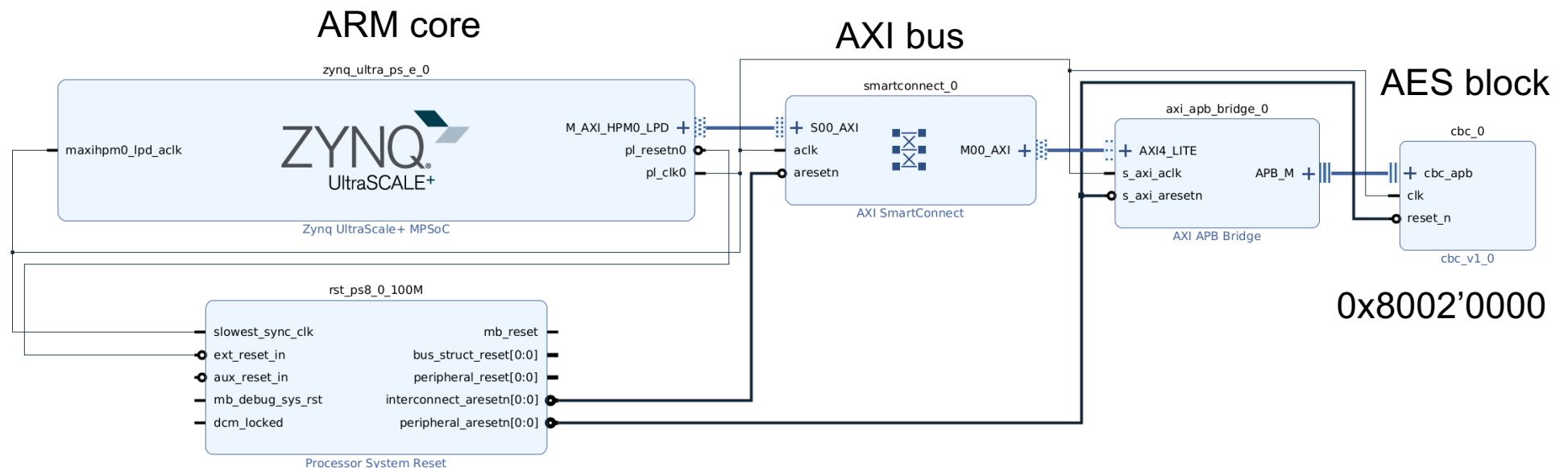
CPU–FPGA Interface

- How do CPU and FPGA blocks communicate?
- AXI (Advanced eXtensible Interface) is an ARM bus protocol for on-chip communication
 - reachable via System Bus
 - Memory mapped: Starting address and range, with its own memory map to blocks/registers in FPGA
 - implement control and status registers in FPGA blocks



AES cryptography in the FPGA

- Zynq: configuration of ARM processor, clocks, interfaces, peripherals
- AXI used for connection: Zynq – ... – AXI-APB bridge – CBC (AES-block)
- APB: ARM Peripheral Bus used to attach peripheral blocks that use control/status registers



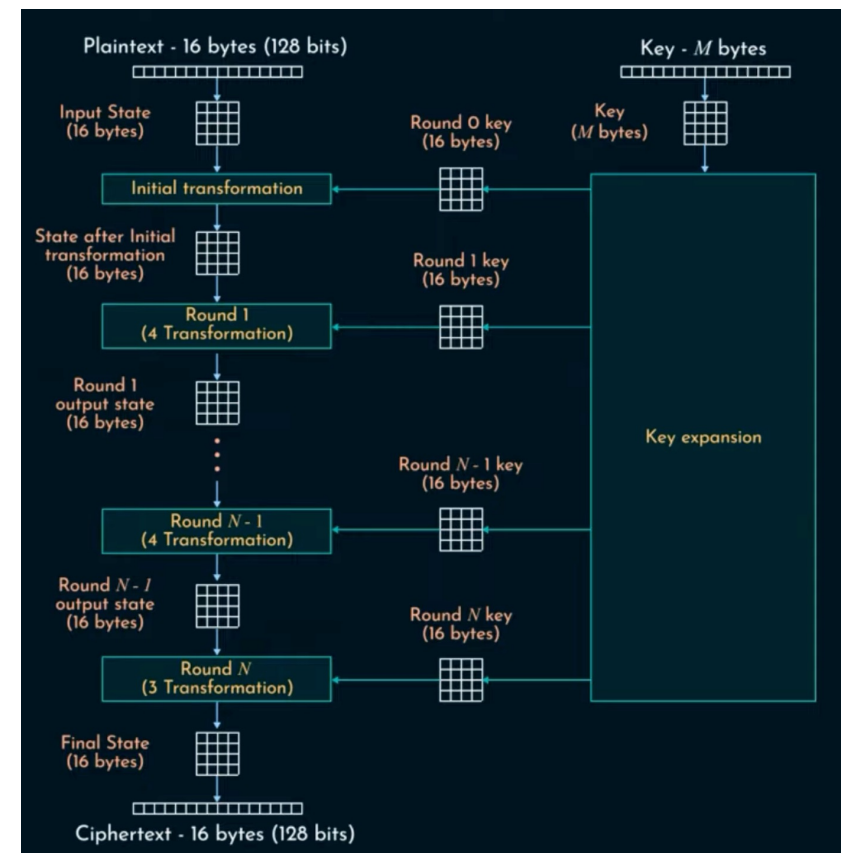
Interaction of CPU and FPGA

- AES CBC implemented as a peripheral block reachable under its own user-defined memory region
- Uses Control and Status registers
- Linux uses managed memory to prevent unauthorized access.
 - ➔ ask Kernel for permission to access memory region of the CBC
 - ➔ receive pointer to the memory region (virtual address), access registers
- Details about access and Register map provided in the Lab instructions

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
zync_ultra_ps_e_0					
zync_ultra_ps_e_0/Data (40 address bits : 0x0080000000 [512M])					
/cbc_0/cbc_apb	cbc_apb	reg	0x00_8002_0000	64K	0x00_8002_FFFF

AES in the FPGA: Why?

- simple operations applied to 128 Bits of data
- Multiple rounds of the same set of operations
 - 10 rounds for 128 bit key
 - 12 rounds for 192 bit key
 - 14 rounds for 256 bit key
- Per round:
 - Substitute Bytes (logic operation)
 - Shift Rows (data reordering)
 - Mix Columns (data reordering)
 - Add Round Key (logic operation)
- AES is designed to be fast, yet hard to crack
- AES explained:
 - <https://www.youtube.com/watch?v=O4xNJsitN6E>
 - <https://www.youtube.com/watch?v=gP4PqVGudtq>



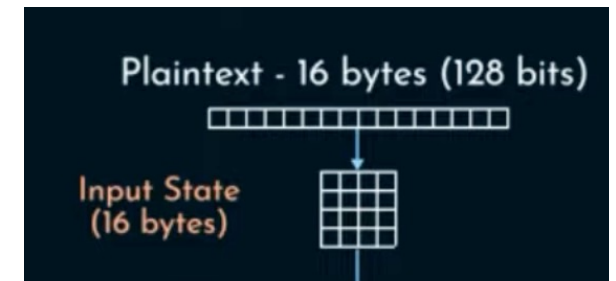
AES step 1: Substitute Bytes

Substitute Bytes

- Each byte is substituted following a calculation in the GF(256)

$$b'_i = \tilde{b}_i \oplus \tilde{b}_{(i+4) \bmod 8} \oplus \tilde{b}_{(i+5) \bmod 8} \oplus \tilde{b}_{(i+6) \bmod 8} \oplus \tilde{b}_{(i+7) \bmod 8} \oplus c_i.$$

- This results in the replacement of each byte with a different byte.
- Suggest a solution for implementation in FPGA:

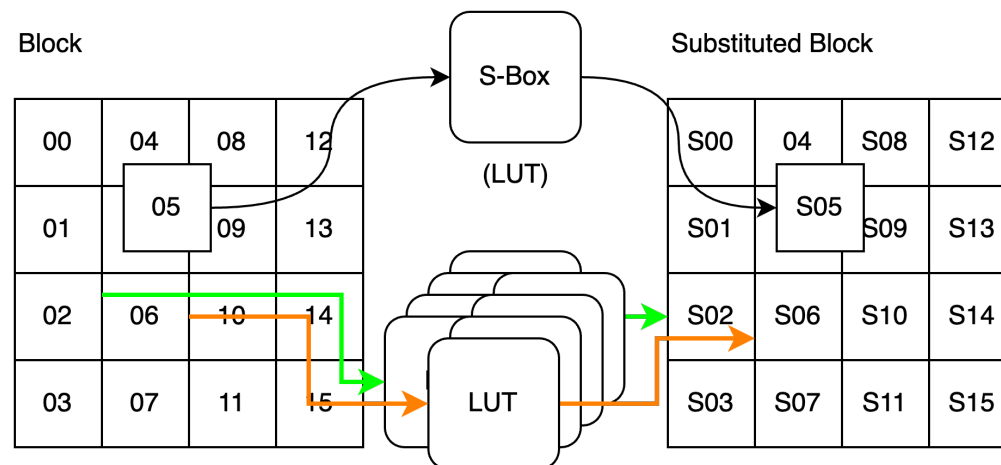


AES step 1 Substitute Bytes

Substitute Bytes

- Each byte is substituted following a calculation in the GF(256)

$$b'_i = \tilde{b}_i \oplus \tilde{b}_{(i+4) \bmod 8} \oplus \tilde{b}_{(i+5) \bmod 8} \oplus \tilde{b}_{(i+6) \bmod 8} \oplus \tilde{b}_{(i+7) \bmod 8} \oplus c_i.$$
- Easier and faster: Lookup Table → ideal for FPGA
- How much faster than a CPU? Single clock cycle for the whole block



S-Box	
In	Out
0x00	0x65
0x01	0x82
0x02	0xF2
0x03	0xCC
...	...

AES step 2 Shift Rows

Shift Rows

- Each row of bytes is shifted by one more byte.
- Software implementation?
- Suggest an FPGA implementation:

Block

00	04	08	12
01	05	09	13
02	06	10	14
03	07	11	15

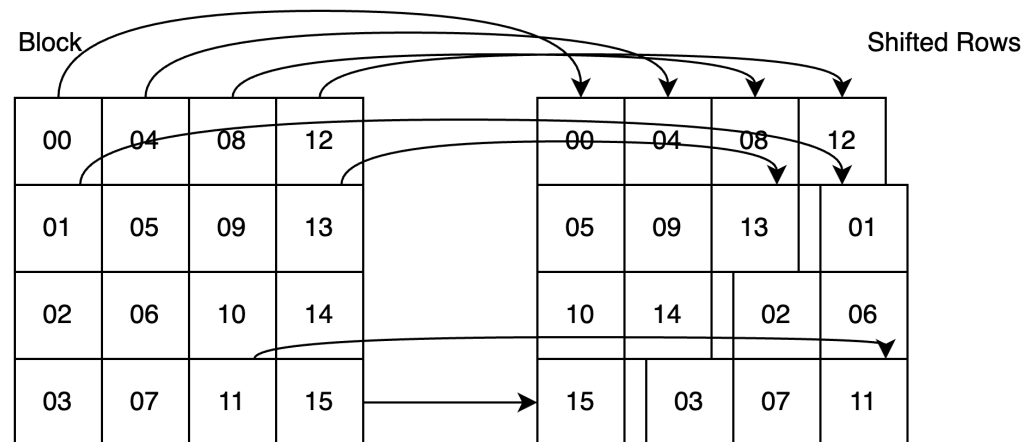
Shifted Rows

00	04	08	12
05	09	13	01
10	14	02	06
15	03	07	11

AES step 2 Shift Rows

Shift Rows

- Each row of bytes is shifted.
- In software, this can be done using a rotation operation (shift with wraparound).
- In software, one instruction per row.
- Pure data moving → can be done routing → ideal for an FPGA
- No clock cycles needed!



AES step 3: Mix columns

Mix Columns

- Each column undergoes a matrix multiplication in GF(256)
- Solution in software?
- Suggest a solution for an FPGA

Matrix

2	3	1	1
1	2	3	1
1	1	2	3
3	1	1	2

*

Block

00	04	08	12
01	05	09	13
02	06	10	14
03	07	11	15

=

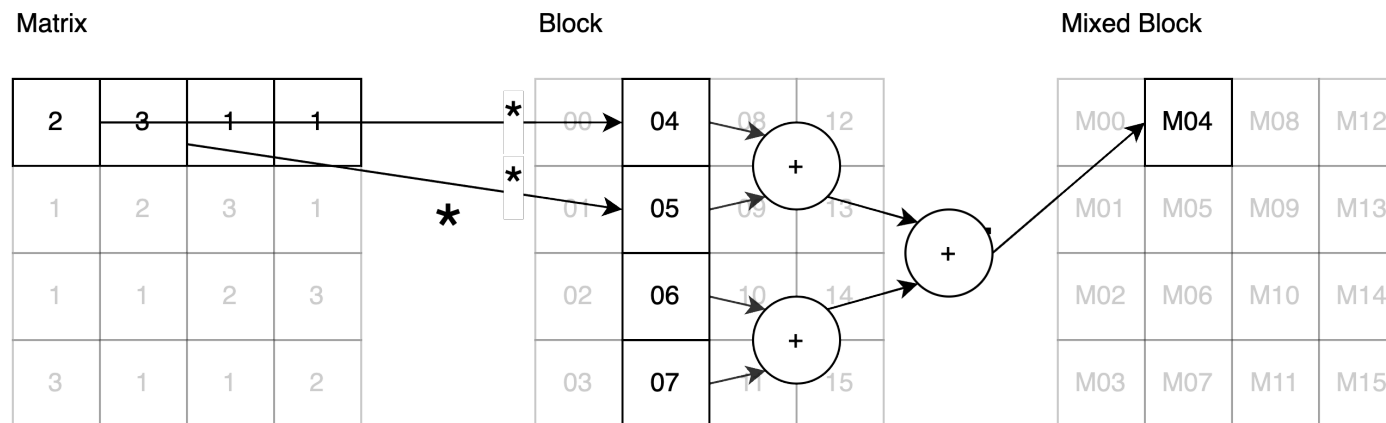
Mixed Block

M00	M04	M08	M12
M01	M05	M09	M13
M02	M06	M10	M14
M03	M07	M11	M15

AES step 3: Mix columns

Mix Columns

- Each column undergoes a matrix multiplication in GF(256)
- Software needs multiple instructions for matrix multiplication.
- FPGA: Parallel execution for each Byte
- Can be done in a single cycle



AES step 4: XOR with Round Key

Round Key XOR

- 16 Bytes of Plaintext arranged in 4 x 4 matrix
- XOR operation with Round Key
- Solution in software?
- Suggest an FPGA implementation:

Block

00	04	08	12
01	05	09	13
02	06	10	14
03	07	11	15



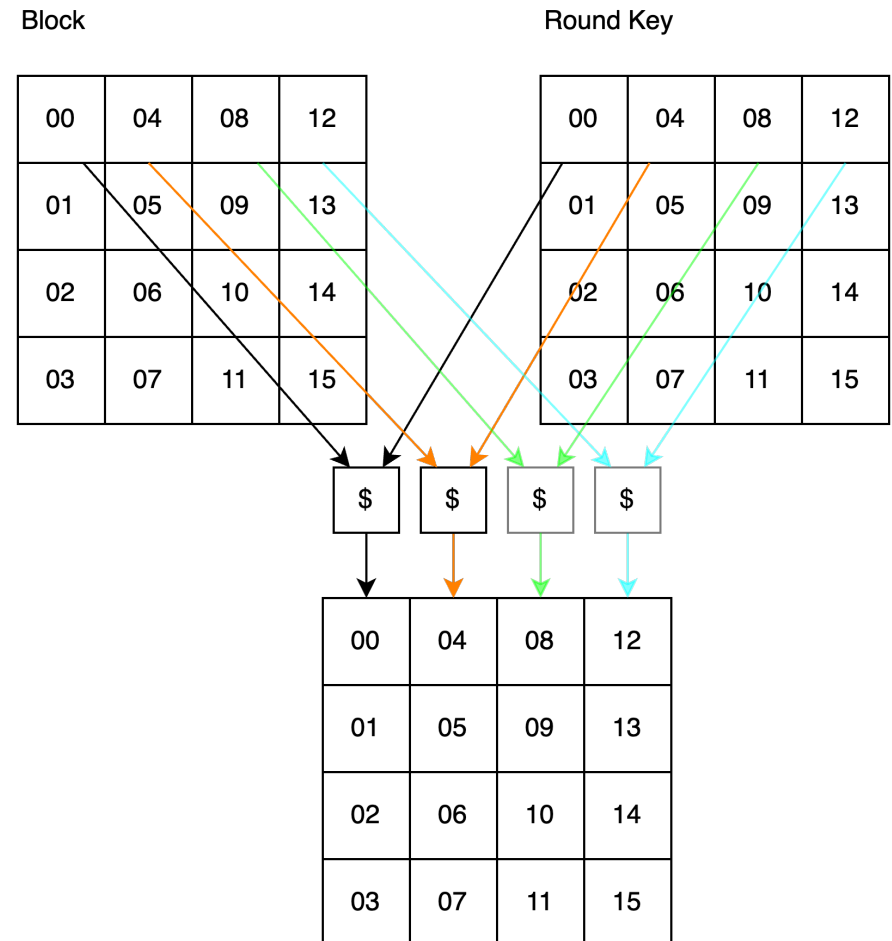
Round Key

00	04	08	12
01	05	09	13
02	06	10	14
03	07	11	15

AES step 4: XOR with Round Key

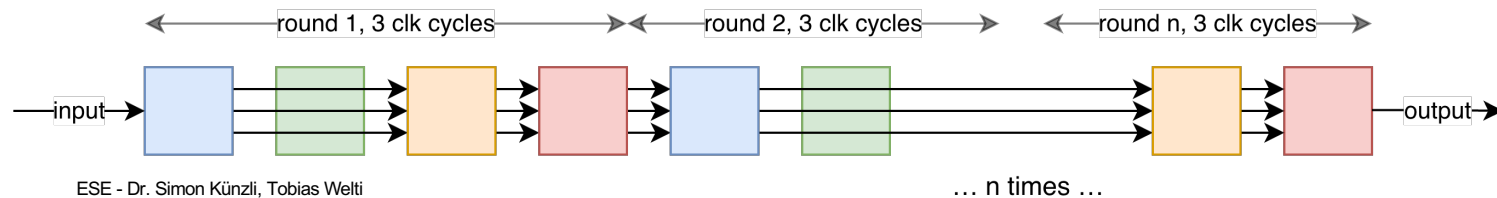
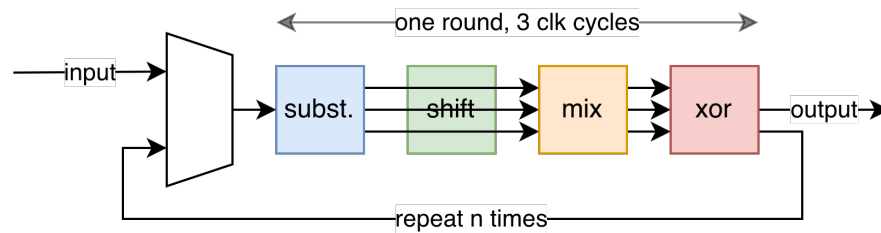
Round Key XOR

- 16 Bytes of Plaintext arranged in 4 x 4 matrix.
- XOR operation with Round Key.
- In software: one XOR operation per row.
- pure bitwise XOR operation → ideal for FPGA
- One clock cycle for a block!



AES in the FPGA: multiple options

- Repetitive operations (loops, rounds) can be executed
 - in sequence: finish on block, then fetch next block
 - pipelined: fetch one block every cycle
- Which one is better?



AES in the FPGA: multiple options

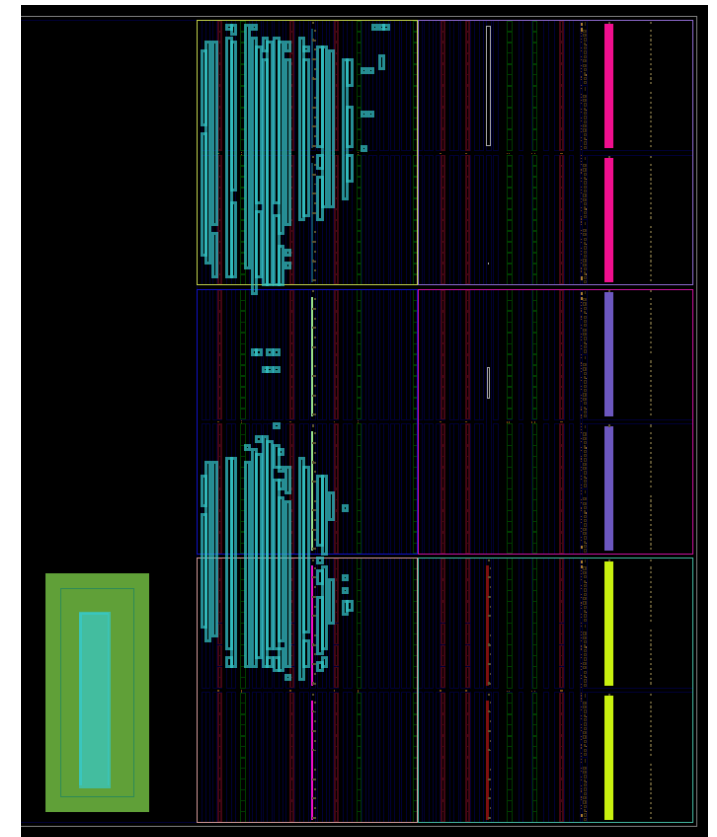
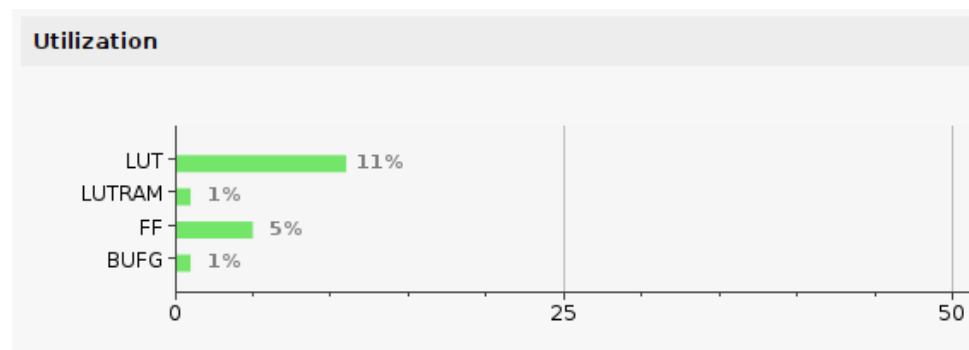
- Repetitive operations (rounds) can be executed
 - in sequence
 - or pipelined
- Which one is better?
- **It depends!**
- Resources vs Performance

AES in the FPGA: multiple options

- Repetitive operations (loops, rounds) can be executed
 - in sequence: finish on block, then fetch next block
 - pipelined: fetch a new block after every cycle
(or after 3 cycles, some AES modes have dependencies on previous results)
- Which one is better?
- **It depends!**
- Resources vs Performance

FPGA Resource usage

- Two AES instances in the ultra96-FPGA use
 - 11 % of available LUT
 - 5 % of Flip Flops
- With (10,12,14) Rounds executed in sequence



Summary

- FPGAs have configurable logic cells than can be interconnected by routing to execute complex logic functions
- Operations that can be parallelized or pipelined can be accelerated in FPGA
- Logic functions and data transfers are good operations for FGPA
- Math operations ($*$, $/$) are more difficult to solve on FPGA than on CPU
- Clock speeds are limited depending on complexity of design
- FPGA development can take more time than SW development
- FPGA often has higher price tag than microcontroller